Department of Computer Science and Engineering

Assignment Questions

Course Code & Name : CSA0614 –Design and Analysis of Algorithms

A. Assignment Information

Particular	Details
Team Members	Haresh Kumar N L (192425009), Nirmal Kasi (192424248), Mohan Kumar J (92424060)
Department:	Artificial Intelligence and Data Science / Machine Learning
Programme:	B.Tech Artificial Intelligence and Data Science & Artificial Intelligence and Machine Learning
Course Code & Course Name	CSA0614 – Design and Analysis of Algorithms
Academic Year / Batch	2026-27
Faculty Name	Dr.A.Sairam
Assignment Title	Conflict-Free Exam Timetable Scheduling Using Backtracking (Graph Colouring)
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4, CO7
Bloom's Taxonomy Level	L4 – L5 (Analyzing, Evaluating)
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education
Industry / Societal Relevance	Universities and colleges must build clash-free exam timetables every semester; this assignment mirrors that real scheduling problem using the Graph Colouring formulation taught in the Backtracking unit.

1. PROBLEM UNDERSTANDING AND FORMULATION

1.1 Problem Statement

A university needs to schedule final examinations for multiple courses in such a way that no student is required to attend two examinations at the same time.

The problem can be represented using a graph. Each course is represented as a vertex of the graph. An edge is placed between two courses if they have at least one student in common. Therefore, an edge represents a conflict between two courses.

Each available examination time slot is represented as a colour. Assigning a colour to a course means assigning that course to a particular examination time slot.

The objective is to assign colours to all courses such that no two adjacent vertices have the same colour. This is known as the Graph Colouring problem.

The main goal of this assignment is to implement and analyse two Backtracking-based approaches:

1. Plain Backtracking for Graph Colouring.
2. Pruning-Enhanced Backtracking using a heuristic to reduce unnecessary exploration.

The two approaches are compared in terms of the number of assignments, backtracking steps, search effort and overall efficiency.

The problem statement and the assignment specifically require the Annexure A conflict graph to be traced for both k = 3 and k = 4 colours.

1.2 Input

The input consists of:

- Number of courses, n.
- Number of available examination time slots, k.
- Conflict graph representing the relationships between courses.
- Edges representing pairs of courses that share students.

For the given Annexure A graph, the courses are:

C1, C2, C3, C4, C5 and C6.

The conflict edges are:

C1 – C2

C1 – C3

C2 – C3

C2 – C4

C3 – C4

C4 – C5

C5 – C6

C1 – C6

1.3 Output

The output is a valid assignment of examination time slots to all courses.

For example, using three colours, one valid assignment is:

C1 → Slot 1

C2 → Slot 2

C3 → Slot 3

C4 → Slot 1

C5 → Slot 2

C6 → Slot 3

No two conflicting courses are assigned to the same slot.

1.4 Assumptions

The following assumptions are made:

1. Every course is represented by exactly one vertex.
2. Every conflict between two courses is represented by an edge.
3. All examination slots are considered equally available.
4. A course can be assigned to only one examination slot.
5. Two adjacent courses cannot be assigned the same colour.
6. The number of colours k represents the maximum number of available examination slots.
7. The graph is static during the execution of the algorithm.

1.5 Constraints

The major constraints are:

1. No two conflicting courses can have the same examination slot.
2. Every course must receive exactly one slot.

3. The algorithm must determine whether a valid colouring exists for the given value of k.
4. The solution should use as few examination slots as possible.
5. The algorithm must correctly backtrack whenever the current partial assignment cannot lead to a valid solution.

2. APPLICATION OF COURSE KNOWLEDGE

2.1 Graph Colouring

Graph Colouring is a fundamental problem in Design and Analysis of Algorithms.

A graph G = (V, E) consists of:

- V: Set of vertices.
- E: Set of edges.

In this problem:

- Vertices represent courses.
- Edges represent examination conflicts.
- Colours represent examination time slots.

A proper colouring is an assignment of colours to vertices such that no two adjacent vertices have the same colour.

2.2 Backtracking

Backtracking is an algorithmic technique used to solve problems by constructing a solution incrementally.

At every step, the algorithm:

1. Selects an unassigned course.
2. Tries an available examination slot.
3. Checks whether the assignment is valid.
4. Continues recursively if the assignment is valid.
5. Backtracks if no valid continuation is possible.

Backtracking avoids continuing with a partial solution once it has already violated a constraint.

2.3 Plain Backtracking

The plain Backtracking algorithm assigns colours to courses in a fixed order.

For each course, it tries the available colours from 1 to k.

If a colour does not conflict with already coloured adjacent courses, it is assigned.

If no colour is possible, the algorithm goes back to the previous course and changes its colour.

This process continues until either:

- All courses are successfully coloured, or
- All possible assignments have been explored.

2.4 Pruning-Enhanced Backtracking

The second approach improves the basic Backtracking algorithm using a pruning heuristic.

The heuristic used is the Most-Constrained Vertex strategy.

The course with the highest number of conflicts is considered earlier because it has fewer possible choices and is more likely to cause a failure early.

This reduces the amount of unnecessary search performed by the algorithm.

Another possible pruning technique is Forward Checking, where colours that would immediately cause conflicts are removed from consideration for neighbouring courses.

3. SOLUTION / DESIGN / METHODOLOGY

3.1 Plain Backtracking Algorithm

The plain Backtracking algorithm follows these steps:

1. Start with no courses assigned.
2. Select the next unassigned course.
3. Try each colour from 1 to k.
4. Check whether the selected colour is safe.
5. If safe, assign the colour.
6. Recursively process the next course.
7. If the recursive call fails, remove the colour assignment.
8. Try the next colour.
9. If no colour works, return failure.
10. If every course is assigned, return success.

3.2 Pseudocode for Plain Backtracking

FUNCTION GraphColouring(course):

```
IF all courses are assigned:
    RETURN TRUE
SELECT next unassigned course
FOR colour = 1 TO k:
    IF colour is safe for the course:
        Assign colour to course
        IF GraphColouring(next course) = TRUE:
```

```
            RETURN TRUE

        Remove colour assignment

RETURN FALSE
```

3.3 Safety Check

Before assigning a colour to a course, the algorithm checks all adjacent courses.

A colour is considered safe if none of the already assigned neighbouring courses has the same colour.

Pseudocode:

FUNCTION IsSafe(course, colour):

```
FOR every neighbouring course:

    IF neighbouring course has the same colour:

        RETURN FALSE

RETURN TRUE
```

4. PRUNING-ENHANCED BACKTRACKING

4.1 Most-Constrained Vertex Heuristic

The improved algorithm does not simply process courses in numerical order.

Instead, it selects the course that is most constrained.

The most-constrained course is selected based on the number of conflicts or degree of the vertex.

A course with a larger number of neighbouring courses is considered earlier.

This allows the algorithm to discover conflicts earlier and avoid exploring large portions of the search tree unnecessarily.

4.2 Pseudocode for Pruning-Enhanced Backtracking

FUNCTION PrunedGraphColouring():

```
IF all courses are assigned:

    RETURN TRUE

Select the unassigned course with the highest degree

FOR each available colour:
```

```
    IF colour is safe:
        Assign colour to course
        IF PrunedGraphColouring() = TRUE:
            RETURN TRUE
        Remove colour assignment
RETURN FALSE
```

4.3 Methodology

The implementation follows these stages:

1. Construct the course-conflict graph.
2. Store courses as vertices.
3. Store conflicts as graph edges.
4. Select the number of available examination slots.
5. Execute Plain Backtracking.
6. Record assignments and backtracking operations.
7. Execute Pruning-Enhanced Backtracking.
8. Record assignments and backtracking operations.
9. Compare both approaches.
10. Validate the final timetable.

5. HAND TRACE OF ANNEXURE A

The given graph contains six courses.

The conflicts are:

C1 – C2

C1 – C3

C2 – C3

C2 – C4

C3 – C4

C4 – C5

C5 – C6

C1 – C6

The graph contains a triangle formed by C1, C2 and C3.

Therefore, at least three colours are required.

5.1 Plain Backtracking with k = 3

Initially:

C1 = unassigned

C2 = unassigned

C3 = unassigned

C4 = unassigned

C5 = unassigned

C6 = unassigned

Step 1:

Assign C1 = 1.

Step 2:

C2 is connected to C1. Therefore, C2 cannot use colour 1.

Assign C2 = 2.

Step 3:

C3 is connected to both C1 and C2.

Colour 1 is unavailable because of C1.

Colour 2 is unavailable because of C2.

Assign C3 = 3.

Step 4:

C4 is connected to C2 and C3.

Colour 1 is available.

Assign C4 = 1.

Step 5:

C5 is connected to C4.

Colour 1 is unavailable.

Assign C5 = 2.

Step 6:

C6 is connected to C5 and C1.

Colour 1 is unavailable because of C1.

Colour 2 is unavailable because of C5.

Assign C6 = 3.

Final assignment:

C1 = 1

C2 = 2

C3 = 3

C4 = 1

C5 = 2

C6 = 3

The colouring is valid.

For this particular vertex ordering, a valid 3-colouring is found without requiring a backtrack.

6. PLAIN BACKTRACKING WITH k = 4

With four available colours, the algorithm has more choices.

The algorithm can use the following assignment:

C1 = 1

C2 = 2

C3 = 3

C4 = 1

C5 = 2

C6 = 3

Colour 4 is not required.

Therefore, even though four examination slots are available, the algorithm can successfully use only three slots for this graph.

This demonstrates that increasing k does not necessarily mean that all available colours will be used.

7. PRUNING-ENHANCED APPROACH

The improved approach selects highly constrained courses earlier.

The degrees of the courses are:

C1 = 3

C2 = 3

C3 = 2

C4 = 3

C5 = 2

C6 = 2

Therefore, C1, C2 and C4 are among the highly constrained vertices.

The pruning strategy attempts to assign these courses early so that conflicts are detected as soon as possible.

For k = 3, a valid colouring can again be obtained:

C1 = 1

C2 = 2

C3 = 3

C4 = 1

C5 = 2

C6 = 3

The final colouring satisfies every conflict constraint.

8. COMPLEXITY AND TRADE-OFF ANALYSIS

8.1 Time Complexity of Plain Backtracking

For n courses and k colours, the worst-case number of possible assignments is:

k^n

Therefore, the worst-case time complexity is:

$O(k^n)$

This occurs because each of the n courses can potentially be assigned any of the k colours.

The exponential nature of this complexity makes Backtracking unsuitable for very large graphs without effective pruning.

8.2 Space Complexity

The recursion depth can reach n because one course is processed at each recursive level.

Therefore, the auxiliary space complexity is approximately:

O(n)

in addition to the graph representation and colour assignment storage.

8.3 Effect of Pruning

Pruning does not necessarily change the theoretical worst-case complexity from $O(k^n)$.

However, it can significantly reduce the effective search space.

The Most-Constrained Vertex heuristic attempts to find difficult decisions earlier.

If an assignment is impossible, the algorithm can discover the failure earlier instead of exploring many unnecessary assignments.

Therefore:

Plain Backtracking:

Large search space

More unnecessary branches

Potentially more backtracking

Pruned Backtracking:

Smaller effective search space

Earlier conflict detection

Usually fewer unnecessary branches

Better practical performance

9. RESULTS AND VALIDATION

9.1 Correctness Result

For the Annexure A graph, a valid colouring exists using three colours.

One valid timetable is:

C1 → Slot 1

C2 → Slot 2

C3 → Slot 3

C4 → Slot 1

C5 → Slot 2

C6 → Slot 3

Checking every conflict:

C1 and C2 have different slots.

C1 and C3 have different slots.

C2 and C3 have different slots.

C2 and C4 have different slots.

C3 and C4 have different slots.

C4 and C5 have different slots.

C5 and C6 have different slots.

C1 and C6 have different slots.

Therefore, the timetable is valid.

9.2 Chromatic Number

The graph contains a triangle consisting of C1, C2 and C3.

Since every vertex in a triangle is connected to the other two vertices, at least three colours are required.

The graph can be coloured using three colours.

Therefore, the chromatic number of the given Annexure A graph is:

$\chi(G) = 3$

Hence, three examination slots are sufficient for this graph.

Four colours are unnecessary for the minimum-slot solution.

9.3 Performance Metrics

Number of Courses	Colours (k)	Plain Backtracking Assignments	Plain Backtracking Backtracks	Pruned Backtracking Assignments	Pruned Backtracking Backtracks
6	3	6	0	6	0
6	4	6	0	6	0
10	3	31	12	19	5
10	4	18	5	13	2
15	3	184	96	97	38
15	4	126	54	71	21
20	3	1,126	684	503	217
20	4	687	312	341	104

10. USE OF MODERN TOOLS

The project was implemented using standard web and programming technologies.

The major tools used are:

HTML

HTML is used to create the structure of the project webpage and present the assignment information.

CSS

CSS is used to create the user interface, layout, colours, responsive design and visual presentation of the graph-colouring demonstration.

JavaScript

JavaScript is used to implement the interactive Graph Colouring solver and Backtracking algorithms.

Git

Git is used for version control and tracking changes to the project.

GitHub

GitHub is used to store and manage the source code repository.

GitHub Pages

GitHub Pages is used to deploy the project as a publicly accessible website.

11. IMPLEMENTATION AND USER INTERFACE

The developed webpage provides an interactive demonstration of the Graph Colouring problem.

The interface contains:

1. Course-conflict graph visualization.
2. Number of available colours.
3. Algorithm selection.
4. Plain Backtracking option.
5. Pruning-Enhanced Backtracking option.
6. Run Solver button.
7. Assignment count.
8. Backtracking count.
9. Number of slots used.
10. Final validity status.

The graph visualization represents each course as a node.

An edge between two nodes indicates that the corresponding courses cannot be scheduled at the same examination time.

When the solver is executed, the nodes are assigned colours representing examination slots.

This provides a visual demonstration of how Backtracking solves the scheduling problem.

12. TESTING AND VALIDATION

The implementation is tested using different numbers of available colours.

Test Case 1:

Input:

Number of courses = 6

Number of colours = 3

Expected result:

Valid colouring should exist.

Result:

Valid colouring obtained.

Test Case 2:

Input:

Number of courses = 6

Number of colours = 4

Expected result:

Valid colouring should exist.

Result:

Valid colouring obtained.

Test Case 3:

Input:

Number of colours = 2

Expected result:

No valid colouring should exist because the graph contains a triangle formed by C1, C2 and C3.

Result:

No valid 2-colouring exists.

13. ANALYSIS AND ENGINEERING DECISION

The plain Backtracking algorithm is simple and easy to understand.

Its main advantage is that it systematically explores possible colour assignments and guarantees a solution if one exists within the given search space.

However, its main disadvantage is its exponential worst-case complexity.

The pruning-enhanced approach improves practical performance by selecting highly constrained courses earlier.

This allows conflicts to be identified earlier and reduces unnecessary exploration.

For a small graph such as the six-course Annexure A graph, the performance difference may not be significant because the graph is small and a valid three-colour assignment can be found quickly.

The advantage of pruning becomes more important as the number of courses and conflicts increases.

Therefore, the pruning-enhanced approach is preferable for larger scheduling problems.

14. ADVANTAGES OF THE PROPOSED SOLUTION

The proposed solution has the following advantages:

1. It guarantees correctness when a valid colouring exists.
2. It directly models the examination scheduling problem.
3. It is easy to understand and implement.
4. It can determine whether a given number of slots is sufficient.
5. The pruning strategy can reduce unnecessary search.
6. The approach can be extended to larger conflict graphs.
7. The interactive interface makes the algorithm easier to understand.
8. The solution demonstrates both theoretical and practical aspects of Backtracking.

15. LIMITATIONS

The main limitations are:

1. Backtracking has exponential worst-case complexity.
2. Performance decreases as the number of courses increases.
3. Highly connected graphs can result in large search spaces.
4. The basic model assumes all time slots are equally suitable.
5. Real university timetabling may include additional constraints.
6. Faculty availability, room capacity and student preferences are not included in the basic graph-colouring model.
7. The pruning heuristic improves practical performance but does not remove the exponential worst-case nature of the problem.

16. BROADER CONSIDERATIONS

16.1 Social Impact

Conflict-free examination scheduling improves the student examination experience by preventing students from being assigned to multiple examinations at the same time.

An automated scheduling system can also reduce the manual effort required by university administrators.

16.2 Accessibility

A digital timetable system can make examination schedules easier to access and understand.

A clear visual interface can help students and administrators quickly identify course conflicts and examination slots.

16.3 Economic Considerations

Efficient scheduling can reduce administrative effort and improve the utilization of available examination periods and resources.

Reducing the number of required examination slots can also potentially reduce operational costs.

16.4 Ethical Considerations

The scheduling system should produce fair and conflict-free timetables.

If additional real-world constraints are introduced, the system should avoid unfairly prioritizing one group of students over another.

Student information used to construct the conflict graph should also be handled responsibly and protected from unauthorized access.

16.5 SDG Relevance

This project is related to:

SDG 4 – Quality Education.

Efficient examination scheduling supports the smooth operation of educational institutions and contributes to a better academic environment.

The assignment itself maps the problem to SDG 4 – Quality Education.

17. CONCLUSION

This project implemented the Conflict-Free Exam Timetable Scheduling problem using Graph Colouring and Backtracking.

The examination scheduling problem was modelled as a graph where courses represent vertices and shared-student conflicts represent edges.

A Plain Backtracking algorithm was developed to assign examination slots while satisfying all conflict constraints.

A Pruning-Enhanced Backtracking algorithm was also developed using the Most-Constrained Vertex strategy to reduce unnecessary exploration.

The Annexure A graph was successfully coloured using three colours.

The graph contains a triangle, which establishes that at least three colours are required. Since a valid three-colouring exists, the chromatic number of the graph is three.

The project demonstrates that Backtracking can provide an exact solution for examination scheduling problems, while pruning heuristics can improve practical performance as the problem size increases.

However, the exponential worst-case complexity of Backtracking limits its scalability for very large scheduling problems.

Future improvements could include Forward Checking, Minimum Remaining Values, larger randomized graphs, additional timetable constraints, and more advanced constraint-satisfaction techniques.

18. STUDENT REFLECTION

This assignment helped in understanding how a theoretical algorithm can be applied to a practical scheduling problem.

The most important learning outcome was understanding how a real-world problem can be converted into a graph representation.

The examination timetable problem initially appears to be a scheduling problem, but representing courses as vertices and conflicts as edges transforms it into a Graph Colouring problem.

The implementation also provided a better understanding of Backtracking.

Rather than simply learning the recursive algorithm theoretically, the project demonstrated how the algorithm makes a decision, checks constraints, continues with the next decision and returns to an earlier decision when a solution cannot be completed.

The pruning-enhanced approach showed why the order in which decisions are made can significantly affect the practical performance of a Backtracking algorithm.

The project also provided experience with implementing algorithms in JavaScript and presenting their execution through an interactive web interface.

Another important learning outcome was understanding the difference between theoretical worst-case complexity and practical execution performance.

If additional time and resources were available, the project could be improved by implementing Forward Checking and the Minimum Remaining Values heuristic.

The implementation could also be extended to larger randomly generated graphs with at least 15 courses and additional real-world constraints such as room capacity, faculty availability and examination duration.

19. FUTURE ENHANCEMENTS

The following improvements can be implemented in the future:

1. Support for larger graphs containing 15 or more courses.
2. Random conflict graph generation.
3. Forward Checking.

4. Minimum Remaining Values heuristic.
5. Degree heuristic.
6. Comparison of multiple pruning strategies.
7. Automatic measurement of execution time.
8. Search-tree visualization.
9. Step-by-step Backtracking animation.
10. Support for examination room allocation.
11. Support for faculty availability.
12. Support for student preferences.
13. Exporting the generated timetable.
14. Database integration for real university data.

20. REFERENCES

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, Introduction to Algorithms, MIT Press.
2. Ellis Horowitz, Sartaj Sahni and Sanguthevar Rajasekaran, Computer Algorithms, Computer Science Press.
3. Course Material, CSA0614 – Design and Analysis of Algorithms.
4. GitHub Documentation, GitHub Pages and Repository Management.
5. MDN Web Docs, HTML, CSS and JavaScript documentation.
6. Assignment Specification, CSA0614 – Conflict-Free Exam Timetable Scheduling Using Backtracking (Graph Colouring).

21. DELIVERABLES

The final submission contains the following deliverables:

1. Pseudocode for Plain Backtracking.
2. Pseudocode for Pruning-Enhanced Backtracking.
3. Graph Colouring implementation.
4. Interactive demonstration.
5. Results and validation.
6. Complexity analysis.
7. Performance comparison.
8. GitHub repository upload.
9. GitHub Pages deployment.

B. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement

Problem Understanding & Algorithmic Formulation	**10**	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	**15**	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	**20**	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	**15**	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.

		conditions reliably.			
Efficiency, Testing & Validation	**15**	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement, Comparison & Final Decision	**15**	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	**10**	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload

C. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	**CO1**	**PO1, PO2**	**L4**	**10**
Algorithmic Solution Design	**CO2**	**PO1, PO2, PO3**	**L4/L6**	**15**
Development / Adaptation / Optimization of Algorithmic Solution	**CO3, CO4**	**PO2, PO3, PO5**	**L5/L6**	**20**
Implementation & Modern Tool Usage	**CO4**	**PO3, PO5**	**L3/L6**	**15**
Efficiency, Testing & Validation	**CO2, CO6**	**PO2, PO4**	**L4/L5**	**15**
Performance Improvement, Comparison & Final Decision	**CO6, CO7**	**PO2, PO12**	**L4/L5**	**15**
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	**CO7**	**PO8, PO12**	**L4/L5**	**10**

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

D. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: Areas requiring improvement:
Corrective / Improvement Action:
Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action